YAML Configuration Guide

This document serves as the instructions and specifications necessary to define Bento Backend API queries in a YAML file.

Introduction

The changes introduced in Bento Backend version 3.8.0 enable the custodian to write a YAML formatted file that defines API queries without having to write any code. The information included in this guide will describe the following:

- The project configuration prerequisites for adding a query definition file.
- The types of queries that can be defined.
- The properties and specifications that can be used to customize the queries in the YAML query definition file.

Pre-requisite

1. **Create an index definition file (YAML format)**

This index file will be used by es_loader.py script to create OpenSearch indexes and load data from Neo4j database to OpenSearch. The examples of file can be found at**:** https://github.com/CBIIT/icdc-dataloader.git

- The Neo4j database containing metadata needs to be exported to OpenSearch

Following properties are needed in the index definition file.

<table>
<tr><th>Property Name</th><th>Description</th></tr>
<tr><td>index_name</td><td>Declare a name of index in OpenSearch. The index is used to store the documents in dedicated data structures in OpenSearch database.
The example of index_name
index_name: subject</td></tr>
<tr><td>type</td><td>Declare a type of data-source. This descries the location where data is being imported from.
Supported types
<table>
<tr><th>Type</th><th>Description</th></tr>
<tr><td>neo4j</td><td>Index data imported from Neo4j</td></tr>
<tr><td>about_file</td><td>Index data imported from an about page definition (YAML) file</td></tr>
</table></td></tr>
</table>

<table>
<tr><td></td><td><table><tr><td>model</td><td>Index data imported from data model (YAML) files</td></tr></table></td></tr>
<tr><td>mapping</td><td>Declare a list of fields and their OpenSearch types. This explains the metadata of structures and the attributes stored in the desired format.
<table><tr><th>Name</th><th>Description</th></tr><tr><td>field_name</td><td>Declare a name of field</td></tr><tr><td>type</td><td>Declare a data type of the field
e.g. keyword, integer, long, double, object, text, nested</td></tr></table>
The example of mapping
<pre>mapping:
  file_id:
    type: keyword
  age:
    type: keyword</pre></td></tr>
<tr><td>cypher_query</td><td>Neo4j graph query that lets you retrieve data. This fields described in the mapping must match return fields in the Cypher query.
The example of cypher_query
<pre>MATCH (ss:study_subject)<-[*..2]-(parent)<--(f:file)
  RETURN DISTINCT
  f.file_id AS file_id,
  f.file_name AS file_name</pre></td></tr>
</table>

The example of index definition files can be found at https://github.com/CBIIT/bento-RI-backend/blob/4.8.0/src/main/resources/yaml/es_indices_bento.yml

```
Indices:
 - index_name: subject
   type: neo4j
   mapping:
     file_id:
       type: keyword
     file_name:
       type: keyword
  cypher_query: "
     MATCH (ss:study_subject)<-[*..2]-(parent)<--(f:file)
     RETURN DISTINCT
     f.file_id AS file_id,
     f.file_name AS file_name
```

"

2. Create query definition files (YAML format). The examples of file can be found at: https://github.com/CBIIT/bento-RI-backend/tree/4.8.0/src/main/resources/yaml

File Type	Description
single_search_es.yml	Defines simple queries
facet_search_es.yml	Defines facet filter queries
global_search_es.yml	Defines global search queries

Search API query definition in Bento-Backend

Property	Description
index	Must declare OpenSearch index name(s). You can use multiple names. e.g. ***index***: - subjects

name	Must match the query name defined in GraphQL schema.
	*The example of **YAML script***
	- ***name***: ***findSubjectIdsInList***
	index:
	- subjects
	filter:
	type: default
	result:
	type: object_array
	
	*The example of **GraphQL schema***
	…
	findSubjectIdsInList(subject_ids: [String] = []): [SubjectResult]
	type SubjectResult {
	subject_id: String
	program_id: String
	}

filter	Declare a supported filter type 'See Table FY-1.0
result	Declare desired return type. See Table RT-1.0

Filter Type Table - FY-1.0

Filter Type	Description

default

By comparing a selectedField and search terms, matching document are filtered.

Name	Required	Description
caseInsensitive	No	Both lowercase and uppercase letters are treated to search for the same.
ignoreIfEmpty	No	As a special case, when all specified fields are empty, the maximum number of data that can be retrieved is returned.

An example of **Default filter with** object_array

YAML Query

```
filter:
  type: pagination
  defaultSortField: file_name
  ignoreIfEmpty:
    - file_ids
```

GraphQL Query

```
query filesInList(
        file_ids: ["BENTO-FILE-0001"],
        first: 1,
        offset: 0,
        order_by: "file_name",
        sort_direction: "desc"
    ) {
        filesInList {
            study_code
            subject_id
            file_name
        }
}
```

Response

```
{
    "data": {
        "filesInList": [
            {
                "study_code": "A",
                "subject_id": "BENTO-CASE-0001",
                "file_name": "1001_bento.txt",
                ….
            }
        ]
    }
}
```

An example of **Default filter(**ignoreIfEmpty) **with** object_array

YAML Query

```
filter:
  type: pagination
```

<table>
<tr><td></td><td>
<table>
<tr><td></td><td>defaultSortField: file_name
ignoreIfEmpty:
- file_ids</td></tr>
<tr><td>GraphQL Query</td><td># If file_ids has no element in array, it returns the maximum number of data.
query filesInList(
file_ids: [],
first: 1,
offset: 0,
order_by: "file_name",
sort_direction: "desc"
) {
filesInList {
study_code
subject_id
file_name
}
}</td></tr>
<tr><td>Response</td><td>{
"data": {
"filesInList": [
{
"study_code": "A",
"subject_id": "BENTO-CASE-0001",
"file_name": "1001_bento.txt",
….
},
{
"study_code": "A",
"subject_id": "BENTO-CASE-0002",
"file_name": "1002_bento.txt",
….
}
…...
]
}
}</td></tr>
</table>
</td></tr>
<tr><td>aggregation</td><td>Provide you with the count of distinct values in a pre-determined format.
Aggregation builds multiple pieces of analytical information into desired set of data.
<table>
<tr><th>Name</th><th>Required</th><th>Description</th></tr>
<tr><td>selectedField</td><td>Yes</td><td>An aggregation filter that performs statistical analysis must select a field.</td></tr>
</table>
</td></tr>
</table>

	ignoreSelectedField	No	Select a field to exclude filtering

An example of **Aggregation** filter
This indicates that how often the studies data appears statically across the document from the filtering

YAML Query	filter:
	  type: aggregation
	  selectedField: studies
	result:
	  type: group_count

GraphQL Query	query searchSubjects (
	  subject_ids: ["id_001", "id_002", “id_003”],
	  ) {
	  subjectCountByStudy {
	  group
	  subjects
	  }
	}

Response	{
	  "data": {
	  "searchSubjects": {
	  "subjectCountByStudy": [
	  {
	  "group": "C: RS 11-25....",
	  "subjects": 347
	  },
	  {
	  "group": "B: RS 11-25...",
	  "subjects": 310
	  },
	  {
	  "group": "D: RS > 25...",
	  "subjects": 175
	  }
	  ]
	  }
	  }
	}

	Name	Required	Description
pagination	defaultSortField	**No**	Select a field for a sorting, if no sorting parameter available, by default, the defaultSortField is used for a sorting
	alternativeSortField	**No**	Prioritize the field for a sorting, preventing letters/words including numbers 0-9 alphabetically. e.g 01-subject-id, 99-sample-id

rangeFilterFields	**No**	Declare fields to be used as a range filter among the given parameters.	

An example of **Pagination filter** with object_array

YAML Query	filter:
	  type: **pagination**
	  defaultSortField: subject_id_num
	  alternativeSortField:
	  subject_id: subject_id_num
	result:
	  type: object_array

GraphQL Query	query subjectOverView (
	  studies: ["A", "B"],
	  first: 1,
	  offset: 0,
	  order_by: "subject_name",
	  sort_direction: "desc"
	
	  ) {
	  subjectCountByStudy {
	  group
	  subjects
	  }
	}

Response	{
	  "data": {
	  " subjectOverview ": [
	  {
	  "study_code": "A",
	  "subject_id": "BENTO-CASE-0001",
	  ….
	  },
	  {
	  "study_code": "B",
	  "subject_id": "BENTO-CASE-0002",
	  ….
	  }
	
	  ]
	  }
	}

range	Search within numerical boundary

Name	Required	Description
selectedField	Yes	A range filter that performs statistical analysis must select a field.
ignoreSelectedField	No	Select a field to exclude filtering

An example of **range** filter with range

YAML Query	filter:
	  type: **range**
	  selectedField: age_at_index
	  ignoreSelectedField: true
	result:
	  type: range

GraphQL Query	query searchSubjects (
	  subject_ids: ["id_001", "id_002", “id_003”],
	  ) {
	  filterSubjectCountByAge {
	  lowerBound
	  upperBound
	  subjects
	  }
	}

Response	{
	  "data": {
	  "searchSubjects": {
	  "filterSubjectCountByAge": {
	  "lowerBound": 29.0,
	  "upperBound": 85.0,
	  "subjects": 1000
	  }
	  }
	  }
	}

sub_aggregation

Aggregation support metric sub-aggregations. For example, sub-aggregation groups the total number of matched documents per bucket

Name	Require	Description
selectedField	Yes	A sub-aggregation filter that performs statistical analysis must select a field.
subAggSelectedField	Yes	Declare to map the corresponding children field

An example of **sub_aggregation** filter with arm_program

YAML Query	filter: type: **sub_aggregation**

		selectedField: programs subAggSelectedField: study_acronym result: type: arm_program

	GraphQL Query	query searchSubjects (
		subject_ids: ["id_001", "id_002", “id_003”],
		) {
		armsByPrograms {
		program
		caseSize
		children {
		arm
		caseSize
		size
		}
		}

	Response	{
		"armsByPrograms": [
		"my-agg-name": {
		"program": "Bento",
		"caseSize": 1000,
		"children": [
		{
		"arm": "A",
		"caseSize": 100,
		"size": 100
		},
		{
		"arm": "B",
		"caseSize": 50,
		"size": 50
		}
		]
		}
		]
		}

global

Name	Required	Description
defaultSortField	Yes	if there is no sorting parameter, by default, the defaultSortField is used.
searches	Yes	List desired field(s) to be searched from filtering See Table **GS-1.0**
typedSearch	No	See Table **TS-1.0**

Global-Searches Table GS-1.0

Name	Required	Description
field	Yes	Select a desired field from filtering
type	Yes	List desired field(s) to be searched from filtering See Table **GS-1.0**
caseInsensitive	No	Both lowercase and uppercase letters are treated to search for the same.

Type-search Table TS-1.0

Query Type	Description
term	Searches based on a precise value like an userid or a price Optional: integer, boolean
match	Searched text, number or boolean value after text scoring analysis
wildcard	Searches items without knowing the exact words

An example of ***global*** filter with global

YAML Query	filter:
	  type: **global**
	  selectedField: content.paragraph
	  searches:
	  - field: content.paragraph
	  type: match
	highlight:
	  fields:
	  - content.paragraph
	  preTag: $
	  postTag: $
	result:
	  type: global_about

GraphQL Query	query globalSearch (
	  input: “bento”,
	  first: 10,
	  offset: 0
	  ) {
	  about_page{
	  page
	  title
	  type
	  text
	  }
	}

Response	{
	"data": {
	"globalSearch": {
	"about_page": [
	{
	"page": "/resources",
	"title": "Resources",
	"type": "about",
	"text": "/$bento$-docs/)."
	}
	]
	}
	}
	}

Highlight a portion of text in a document that matches the search keyword. Highlighting is very useful to find a term in long texts.

Name	Required	Description
fields	Yes	A list of fields to highlight
fragmentSize	No	The size of the highlighted fragment. Default by 1
preTag	No	Use html tag to wrap before the highlighted text. Default by $
postTag	No	Use html tag to wrap after the highlighted text. Default by $

An example of search terms of “bento”

{
"data": {
"globalSearch": {
"about_page": [
{
"page": "/resources",
"title": "Resources",
"type": "about",
"text": "/***$bento$***-docs/)."
}
]
}
}
}

nested	Nested is a type of object stored in inner documents as arrays of objects to be indexed for a separate document.

The example of field definition for **nested Index**

```
Indices:
 - index_name: subjects
   type: neo4j
   mapping:
     programs:
       type: keyword
     program_id:
       type: keyword
     sample_info:
       type: nested
       properties:
         sample_id:
           type: keyword
         composition:
           type: keyword
     file_info:
       type: nested
       properties:
         file_id:
           type: keyword
         file_name:
           type: keyword
```

The example of stored **nested data** in OpenSearch


```
{
  ...
  "hits": [
    {
      "_index": "subjects",
      "_source": {
        "programs": "TAILORx",
        "subject_id": "BENTO-CASE-0001",
        "sample_info": [
          {
            "composition": "Solid Tissue",
            "sample_id": "BENTO-BIOS-0001"
          },
          {
            "composition": "Soft Tissue",
            "sample_id": "BENTO-BIOS-0002"
          }
        ],
        "file_info": [
          {
            "file_id": "BENTO-FILE-0001",
            "file_name": "0001_bento_pcr.txt"
          },
```

```
        {
          "file_id": " BENTO-FILE-0002",
          "file_name": "0002_bento_pcr.txt"
        }
      ]
    }
  },
  ...
 ]
}
```

The nested filter searches in the nested field objects. If stored document matches the search, the nested filter returns the root parent document

Name	Required	Description
selectedField	Yes	An aggregation filter that performs statistical analysis must select a field.
nestedPath	Yes	Declare a path to the nested object you wish to search An example of ***nested*** *document* Indices: - index_name: subjects type: neo4j mapping: program: type: keyword sample_info: type: ***nested*** properties: sample_id: type: keyword sample_ids: type: keyword
nestedParameters	Yes	Declare parameters to be searched by filtering in the nested path
ignoreSelectedField	No	Field(s) to be excluded from filtering

An example of ***nested*** filter with group_count

YAML Query	filter: type: **nested** selectedField: file_ids nestedPath: file_info nestedParameters: - file_type result:

		type: int method: nested_count

	GraphQL Query	query searchSubjects (
		subject_ids: ["id_001", "id_002", “id_003”],
		) {
		numberOfFiles
		}

	Response	{
		"data": {
		"searchSubjects": {
		"numberOfFiles": 1000
		}
		}
		}

sum

Sum up numerical values from the aggregated search.

Name	Required	Description
selectedField	Yes	A sum filter that performs statistical analysis must select a field.

An example of **sum** filter with float(sum)

YAML Query	filter: type: **sum** selectedField: file_size result: type: float method: sum

GraphQL Query	query searches(
	file_ids: ["bento-file-001", "bento-file-002"]
	) {
	fileSize
	}

Response	{
	"data": {
	"searches": {
	"fileSize": 9999.99
	}
	}
	}

Query Configuration Result Type Table - RT-1.0

Result Type	Optional Method	Example
object_array	-	[object, object, object...]
group_count	-	{boy: 10, girl:20...}
str_array	-	[boy, girl...]
int	count_bucket_keys, nested_count	1, 2, 3 ...

	see Table **OM-1.0**	
float	sum see Table **OM-1.0**	1.0, 2.0, 3.0...
global_about	**-**	{type: '', page: '', title: '', text: ''}
global	**-**	{boy: object, girl: object}

Optional Method Table - OM-1.0

Optional Method	Description
count_bucket_keys	Return an integer for the result of aggregation filter
nested_count	Return an integer for the result of nested aggregation filter
sum	Return a float; sum of all aggregated data for sum aggregation filter

Query Pairing

A filter type must pair with a result type

Query Type	Result Type
default	object_array, str_array
pagination	object_array
sub_aggregation	arm_program
aggregation	aggregation, int(count_bucket_keys)
range	range
global	global, global_multi_models, global_about
nested	int(nested_count), nested_list, nested_total
sum	float(sum)